

15/12/2023
Friday

MODULE-2

- * Regular Expression's:
 - ⇒ Regular languages:
 - ⇒ Regular Expressions
- * Finite Automata and Regular Expression
- * Applications of Regular Expressions
- * Regular Grammars

Introduction

We define regular expressions as a means of representing certain subsets of strings over Σ and will prove that regular sets are precisely those accepted by automata.

We use pumping lemma for regular sets to prove that certain sets are not regular. Then we discuss about the properties of regular expressions.

Regular Expression

→ A regular expression is the language accepted by finite automata is called regular language.

A regular language can be described using regular expressions, consisting of alphabets in Σ and the operators $'*'$, $'|'$, $'^+'$, $'^o'$. The order of evaluation of regular expression is determined by parentheses and the

Operator precedence $'*'$, $'\cdot'$, $'\cup'$, $'\epsilon'$ respectively.

A Regular expression is recursively defined as follows

✓ $'\emptyset'$ is a regular expression denoting an empty language

✓ $'\epsilon'$ is a regular expression denoting the language containing an empty string

$'a'$ is a regular expression denoting the language containing only $\{a\}$

$\Rightarrow$ If R is a regular expression denoting the language L_R and S is a regular expression denoting the language L_S , then

$\Rightarrow R + S$ is a regular expression corresponding to the language $L_R \cup L_S$.

$\Rightarrow R \cdot S$ is a regular expression corresponding to the language $L_R \cdot L_S$.

$\Rightarrow R^*$ is a regular expression corresponding to the language L_R^* .

The expressions obtained by applying any of these rules are called regular expressions.

Note: Any set represented by a regular expression is called Regular set.

Example:

Let $\Sigma = \{a\}$ and regular expression $R = a^*$ the R is a set denoted by $L = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

Basic Operations on RE

$\Rightarrow$ Concatenation, Union and Kleen closure.

Let R & S be any two RE

$\Rightarrow$ concatenation: Concatenation of regular expressions R and S is $RS = \{xy \mid x \in R \text{ and } y \in S\}$

Ex:-

If $R = \{abc\}$ and $S = \{d, ef\}$ then $RS = \{abd, cd, abef, cef\}$

$\Rightarrow$ union: Union of RE R and S is $R + S = \{x \mid x \in R \text{ or } x \in S\}$

Ex:-

If $R = \{ab, c\}$ and $S = \{d, ef\}$ then $R + S = \{ab, c, d, ef\}$

$\Rightarrow$ Kleene closure: Kleene closure is the set of all strings, that can be made by concatenating zero or more strings in R .

Ex:-

If $R = \{ab, c\}$ the $R^* = \{\epsilon, ab, c, abab, ca, abc, cab, \dots\}$

Properties of Regular Expressions

Let P , Q and R be any arbitrary regular expression.

Then, the following properties are "true"

1 $\epsilon R = R \epsilon = R$

2 $\epsilon^* = \epsilon$

3 $(\emptyset)^* = \epsilon$

4 $\emptyset R = R \emptyset = \emptyset$

5 $\emptyset + R = R$

6 $R + R = R$

7 $RR^* = R^*R = R^*$

8 $(R^*)^* = R^*$

9 $\epsilon + RR^* = R^*$

10 $(P + Q)R = PR + QR$

11 $(P + Q)^* = (P^*Q^*)^* = (P^* + Q^*)^*$

12 $R^*(R^* + R) = (R^* + R)R^* = R^*$

13 $(R + R^*)^* = R^*$

14 $\epsilon + R^* = R^*$

15 $(PQ)^*R = P(QP)^*$

16 $R^*R + R = R^*R$

17 $(PQ)^* = (P^*Q^*)^* = (P^* + Q^*)^*$

18 $R^*R^* = (R^*)^* = R^*$

Describing REREMeaning 0^* Set of strings of 0's of any length including ϵ 0^+ Set of strings of 0's of any length excluding ϵ [atleast one 0] $(0+1)^*$ set of strings of 0's & 1's of any length including ϵ $(0+1)^+$ Set of strings of 0's & 1's of any length excluding ϵ $0+1$

Set of strings consisting of either one 0 or one 1.

 $(0+1)^* 011$

set of strings of 0's and 1's ending with 011

 $01(0+1)^*$

set of strings of 0's & 1's starting with 01

 $0^* 1^* 2^*$ Set of strings of 0's & 1's & 2's including ϵ [in order] $0^+ 1^+ 2^+$ set of strings of any number of 0's followed by any number of 1's followed by any number of 2's excluding ϵ $00^+ 11^* 22^*$

Set of strings of 0's and 1's and 2's with atleast one 0, followed by one 1, followed by one 2.

 $(0+1)^* (0+11)$

set of strings of 0's and 1's ending with either 0 or 11

Problems

- 1) Write the RE for the following language accepting all combinations of a's over the set $\Sigma = \{a\}$

Soln $L = \{ \epsilon, a, aa, aaa, aaaa, \dots \}$
 $\therefore R = a^*$

- 2) Design a RE for the language containing any number of a's and b's

Soln $\Sigma = \{a, b\}$
 $L = \{ \epsilon, a, b, ab, ba, aa, bb, abab, baba, \dots \}$
 $\therefore R = (a+b)^*$

- 3) Construct the RE for the language containing all the strings which ends with 00 over the set $\Sigma = \{0, 1\}$

Soln $\Sigma = \{0, 1\}$
 $L = \{ \epsilon, 00, 000, 100, 1000, 10000, \dots \}$
 $\therefore R = (0+1)^*00$

- 4) Construct the RE for the language consisting of the set of all strings of 0's and 1's beginning with 0 and ending with 1

Soln $\Sigma = \{0, 1\}$

$L = \{01, 001, 011, 0011, 0101, 001011, \dots\}$

$\therefore R = 0(0+1)^*1$

5) Construct a RE for the following language.

$L = \{ \epsilon, 11, 1111, 111111, \dots \}$

$\Sigma = \{1\}$

Any element of L is either ϵ or a string of even number of 1's

$\therefore R = (11)^*$

6) Design the RE for the language that accepts all combinations of a's except the null string given $\Sigma = \{a\}$

$L = \{a, aa, aaa, aaaa, \dots\}$

$\therefore R = a^+$

7) Construct a RE for the language consisting of all strings of a's and b's beginning with a and ending with ab.

$\Sigma = \{a, b\}$

$L = \{aabb, aaab, abab, aabab, aabaab, abbaab, \dots\}$

$\therefore R = a(a+b)^*ab$

8) Construct a RE for the set of strings of 0's and 1's that should contain the substring 000

Soln

$$\Sigma = \{0, 1\}$$

$$L = \{000, 0000, 000001, 1000001, \dots\}$$

$$\therefore R = (0+1)^* 000 (0+1)^*$$

9) Give a RE for representing the set L of strings in which every 0 is immediately followed by atleast two 1's

Soln

$$\Sigma = \{0, 1\}$$

The elements of L is zero or more occurrences of 1 or 0 followed by 11

$$\text{i.e. } L = \{1, 11, 011, 1011, \dots\}$$

$$\therefore R = (1+011)^*$$

10) Construct a RE for the set of strings of a's & b's, ending with either a or bb

Soln

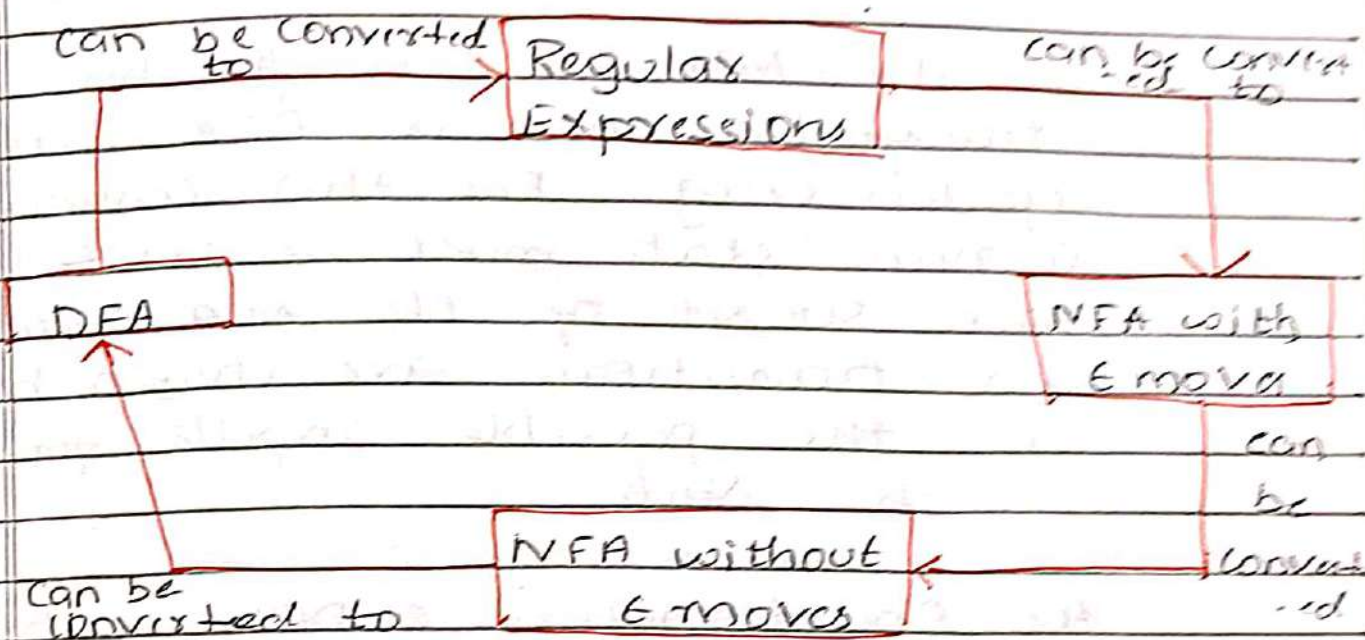
$$\Sigma = \{0, 1\}$$

$$L = \{a, bb, aa, abb, baa, \dots\}$$

$$\therefore R = (a+b)^* (a+bb)$$

Relationship between Finite Automata and Regular Expression

Finite automata and regular expressions are closely related concepts in the theory of computation.



The relationship between Finite Automata and Regular Expressions can be described in several steps.

1) Using epsilon moves, regular expressions can be transformed into Non-deterministic Finite Automata (NFA). Without an input symbol, epsilon moves let you jump from one state to another.

2) The NFA with epsilon moves can be transformed into an equivalent NFA without epsilon moves.

This process involves eliminating the epsilon transitions by performing the epsilon transitions by performing the epsilon closure operation and creating new transitions.

3) The NFA can be further transformed into a DFA without epsilon states. For this conversion, a new state must be created for each subset of the NFA states, and transitions are chosen based on the possible inputs for each state.

4) Additionally, a DFA can be converted back into a RE.

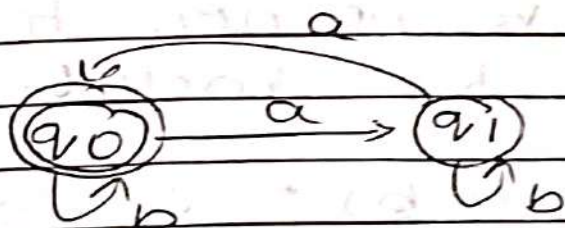
This kind of data conversion is possible using methods like state elimination, in which unnecessary states are gradually eliminated until a RE representation is produced.

Construction of DFA from RE

Let discuss how we can convert RE into Finite automata.

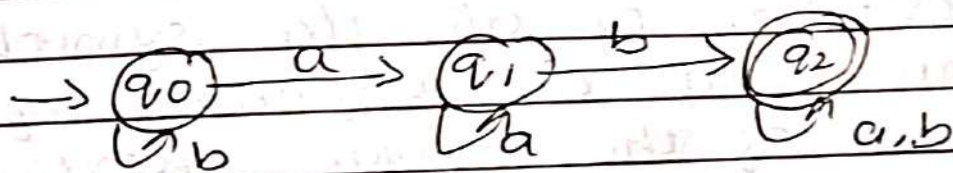
1) Even number of a's

Soln The RE for even number of a's is $(b|ab^*ab^*)$.
We can construct a FA as



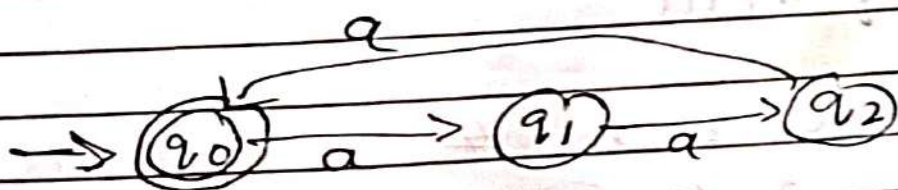
2) String with 'ab' as substring

Soln The RE for strings with 'ab' as substring is $(a|b)^*ab(a|b)^*$



3) String with count of 'a' divisible by 3

Soln The RE for strings with count of a divisible by 3 is $\{a^{3n} | n \geq 0\}$



Problems

- 1) Construct DFA for the regular expression $(a+b)^*abb$

Step 1: Augment the given regular expression with the symbol $\#$

$(a + b)^* a b b \#$

Step 2: Give position to each symbol in the RE including $\#$ symbol

$(a + b)^* a b b \#$

1 2 3 4 5 6

Step 3: Find Firstpos of the given regular expression

Firstpos is a set that contains the positions of all the symbols which can be at the beginning of a valid word of the regular expression

Firstpos becomes the start state of DFA

Firstpos $[(a+b)^*abb\#]$ is $\{1, 2, 3\}$

"1" is included in the Firstpos because of word

$a b a b b \#$ or $a a a b b \#$
1 2 3 4 5 6 1 1 3 4 5 6

2 is included in the Firstpos because of the word

b a b b # or b b a b b #
2 3 4 5 6 2 2 3 4 5 6

or

b a a b b #
2 1 3 4 5 6

3 is included in Firstpos because of the word

a b b #
3 4 5 6

start state of DFA $q_0 = \{1, 2, 3\}$
a, b, a

Step 4: Find followpos of each symbol.
Followpos of a symbol is a set which contain positions of all the symbols which can follow the current symbol.

In the RF

(a + b) * a b b #

1 2 3 4 5 6 .

Followpos(1) = $\{1, 2, 3\}$

Followpos(2) = $\{1, 2, 3\}$

Followpos(3) = $\{4\}$

Followpos(4) = $\{5\}$

Followpos(5) = $\{6\}$

Followpos(6) = $\emptyset$

Step 5: Find set of states of DFA and transitions.

For the start state q_0 , Find transition for $\Sigma = \{a, b\}$.

Since a occurs in first and third position in q_0 ,

$$\begin{aligned} S(q_0, a) &= \text{Followpos}(1) \cup \text{Followpos}(3) \\ &= \{1, 2, 3\} \cup \{4\} \\ &= \{1, 2, 3, 4\} \rightarrow (q_1) \\ &= a b a b \end{aligned}$$

Since b occurs in second position in q_0 ,

$$\begin{aligned} S(q_0, b) &= \text{Followpos}(2) \\ &= \{1, 2, 3\} \rightarrow (q_0) \end{aligned}$$

For the new state q_1 , Find the transition for $\Sigma = \{a, b\}$.

$$\begin{aligned} S(q_1, a) &= \text{Followpos}(1) \cup \text{Followpos}(3) \\ &= \{1, 2, 3\} \cup \{4\} \\ &= \{1, 2, 3, 4\} \rightarrow (q_1) \end{aligned}$$

$$\begin{aligned} S(q_1, b) &= \text{Followpos}(2) \cup \text{Followpos}(4) \\ &= \{1, 2, 3\} \cup \{5\} \\ &= \{1, 2, 3, 5\} \rightarrow (q_2) \\ &= a b a b \end{aligned}$$

For the state q_2 find the transitions

$$\begin{aligned}\delta(q_2, a) &= \text{Followpos}(1) \cup \text{Followpos}(3) \\ &= \{1, 2, 3\} \cup \{4\} \\ &= \{1, 2, 3, 4\} \rightarrow (q_1)\end{aligned}$$

$$\begin{aligned}\delta(q_2, b) &= \text{Followpos}(2) \cup \text{Followpos}(5) \\ &= \{1, 2, 3\} \cup \{6\} \\ &= \{1, 2, 3, 6\} \rightarrow (q_3)\end{aligned}$$

a b a #

For the state q_3 find the transition

$$\begin{aligned}\delta(q_3, a) &= \text{Followpos}(1) \cup \text{Followpos}(3) \\ &= \{1, 2, 3\} \cup \{4\} \\ &= \{1, 2, 3, 4\} \rightarrow (q_1)\end{aligned}$$

$$\begin{aligned}\delta(q_3, b) &= \text{Followpos}(2) \\ &= \{1, 2, 3\} \rightarrow (q_0)\end{aligned}$$

Since there is no, new state, stop the iteration.

Step 6: The DFA M is defined as
 $M = (Q, \Sigma, \delta, q_0, F)$

where $Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$F = \{q_3\}$

Since # occurs in q_3

δ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
* q_3	q_1	q_0

Step 5: Find set of states of DFA and transitions

For start state q_0 find the transitions

Since a occurs at first position in q_0

$$\begin{aligned}\delta(q_0, a) &= \text{Followpos}(1) \\ &= \{2, 4\} \rightarrow \textcircled{q_1} \\ &\quad b \neq\end{aligned}$$

$$\begin{aligned}\delta(q_0, b) &= \text{Followpos}(3) \\ &= \{4\} \rightarrow \textcircled{q_2}\end{aligned}$$

For the state q_1 , find the transition

$$\begin{aligned}\delta(q_1, a) &= \emptyset \\ \delta(q_1, b) &= \text{Followpos}(2) \cup \text{Followpos}(4) \\ &= \{2, 4\} \cup \emptyset \\ &= \{2, 4\} \rightarrow \textcircled{q_1}\end{aligned}$$

For the state q_2 , Find the transition

$$\delta(q_2, a) = \emptyset$$

$$\delta(q_2, b) = \emptyset$$

Step 6: The DFA M is defined as
 $M = (Q, \Sigma, \delta, q_0, F)$

where

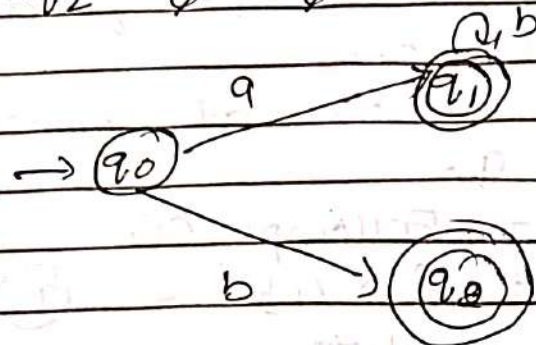
$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

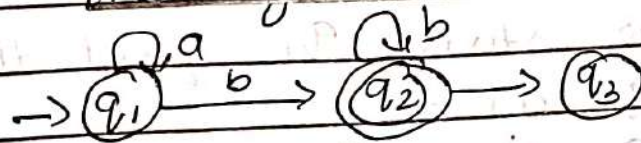
$$q_0 = \{q_0\}$$

$$F = \{q_1, q_2\} \text{ as } \# \text{ occurs in both } q_1 \text{ \& } q_2$$

δ	a	b
$\rightarrow q_0$	q_1	q_2
$* q_1$	$\emptyset$	q_1
$* q_2$	$\emptyset$	$\emptyset$



1) Construct Regular Expression from the given DEA



Soln.

Step 1: q_1 is the start state

Step 2: Calculate q_i such that $q_i = \alpha_{ji} q_j + \epsilon$

where α_{ji} is the transition from q_j to q_i .

$$q_1 = q_1 a + \epsilon \quad [\text{initials}]$$

$$q_1 = \epsilon + q_1 a$$

$$q_1 = \epsilon \cdot a^*$$

$$q_1 = a^*$$

where $q_i \in Q$

$R = Q + RP$ then

$$R = QP^*$$

$$[\because \epsilon a^* = a^*]$$

$$q_2 = q_1 b + q_2 b$$

$$q_2 = a^* b + q_2 b \quad \therefore [q_1 = a^*]$$

$$q_2 = (a^* b) b^*$$

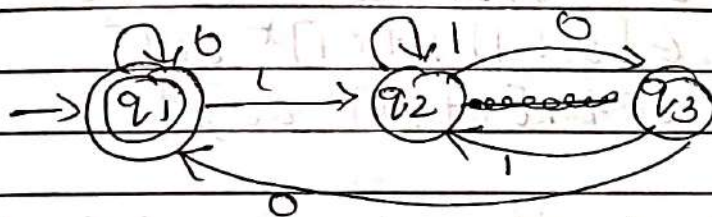
$$q_2 = a^* b b^*$$

$$q_2 = a^* b^+ \quad [\because b \cdot b^* = b^+]$$

We don't want to find out q_3 eqn because q_2 is the final state.

The regular expression is given by
 $a^* b^+$

2) Construct a regular expression from the given DFA



Step 1: q_1 is the start state

Step 2: calculate q_1

$$q_1 = q_1 0 + q_3 0 + \epsilon$$

Step 3: calculate q_2

$$q_2 = q_1 1 + q_2 1 + q_3 1$$

Step 4: calculate q_3

$$q_3 = q_2 0 +$$

Substitute q_3 in q_2

$$q_2 = q_2 \cdot 1 + q_3 \cdot 1 + q_1 \cdot 1$$

$$q_2 = q_2 \cdot 1 + q_2 \cdot 01 + q_1 \cdot 1$$

$$q_2 = q_2 (1 + 01) + q_1 \cdot 1$$

$$q_2 = q_2 \cdot 1 + q_2 (1 + 01)$$

$$q_2 = q_1 (1 + 01)^*$$

Substitute in q_1

$$q_1 = q_1 \cdot 0 + q_3 \cdot 0 + \epsilon$$

$$q_1 = q_1 \cdot 0 + q_2 \cdot 00 + \epsilon$$

$$q_1 = q_1 \cdot 0 + [q_1 (1 + 01)^*] 00 + \epsilon$$

$$q_1 = q_1 [0 + 1(1 + 01)^* 00] + \epsilon$$

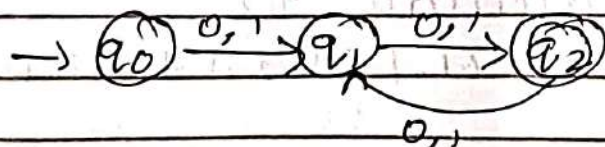
$$q_1 = \epsilon + q_1 [0 + 1(1 + 01)^* 00]$$

$$q_1 = \epsilon [0 + 1(1 + 01)^* 00]^*$$

$$q_1 = [0 + 1(1 + 01)^* 00]^*$$

Since q_1 is the final state the required regular expression is
 $[0 + 1(1 + 01)^* 00]^*$

3. Construct regular expression from the given DFA.



Step 1: q_0 is the start state

Step 2: calculate q_0

$$q_0 = \epsilon$$

Step 3: calculate q_1 and q_2

$$q_1 = q_0 0 + q_0 1 + q_2 0 + q_2 1$$

$$= \epsilon \cdot 0 + \epsilon \cdot 1 + q_2 (0 + 1)$$

$$0 + 1 + q_2 (0 + 1)$$

$$q_2 = q_1 0 + q_1 1$$

$$q_2 = q_1 (0 + 1)$$

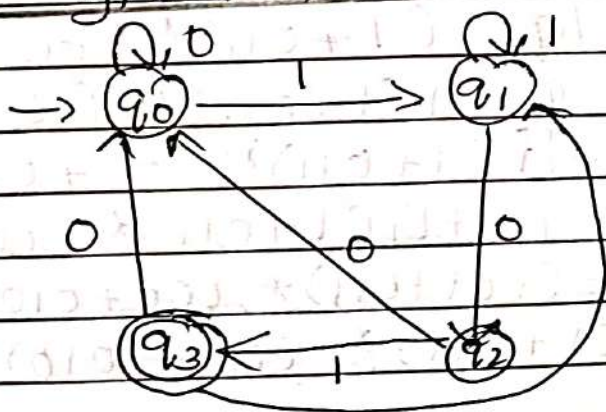
$$q_2 = [0 + 1 + q_2 (0 + 1)] [0 + 1]$$

$$q_2 = [(0 + 1)(0 + 1) + q_2 [(0 + 1)(0 + 1)]]$$

$$q_2 = (0 + 1)(0 + 1)[(0 + 1)(0 + 1)]^*$$

Since q_2 is the final state, the required regular expression is $(0 + 1)(0 + 1)[(0 + 1)(0 + 1)]^*$.

Q. Construct regular expression for the given DFA.



Step 1: q_0 is the start state

Step 2: calculate q_0

$$q_0 = q_0 \cdot 0 + q_2 \cdot 0 + q_3 \cdot 0 + \epsilon$$

Step 3: calculate q_1

$$q_1 = q_0 \cdot 1 + q_1 \cdot 1 + q_3 \cdot 1$$

$$q_2 = q_1 \cdot 0$$

$$q_3 = q_2 \cdot 1$$

Substitute q_2 in q_3

$$q_3 = q_1 \cdot 01$$

$$q_1 = q_0 \cdot 1 + q_1 \cdot 1 + q_3 \cdot 1 \rightarrow q$$

Substitute q_3 in q_1

$$q_1 = q_0 \cdot 1 + q_1 \cdot 1 + q_3 \cdot 1$$

$$q_1 = q_0 \cdot 1 + q_1 \cdot 1 + q_1 \cdot 01$$

$$q_1 = q_0 \cdot 1 + (1 + 011)^* \rightarrow (q_1)$$

process: $q_0 = q_0 \cdot 0 + q_2 \cdot 0 + q_3 \cdot 0 + \epsilon$

$$q_0 = q_0 \cdot 0 + (q_2 \cdot 0) \cdot 0 + (q_1 \cdot 01) \cdot 0 + \epsilon$$

$$q_0 = q_0 \cdot 0 + q_1 [00 + 010] + \epsilon \rightarrow (q_1)$$

Substitute q_1 in the eqn of q_0

$$q_0 = q_0 \cdot 0 + [q_0 \cdot 1 (1 + 011)^* (00 + 010)] + \epsilon$$

$$q_0 = q_0 \cdot 0 + q_0 [1 (1 + 011)^* (00 + 010)] + \epsilon$$

$$q_0 = q_0 [0 + [1 (1 + 011)^* (00 + 010)]] + \epsilon$$

$$q_0 = \epsilon + q_0 [0 + [1 (1 + 011)^* (00 + 010)]]$$

$$q_0 = \epsilon [0 + [1 (1 + 011)^* (00 + 010)]]^*$$

$$q_0 = 0 + (1 + 011)^* (00 + 010)^*$$

Substitute q_1 in q_3

$$q_3 = q_1 \cdot 01$$

$$= q_0 \cdot 1 (1 + 011)^* 01$$

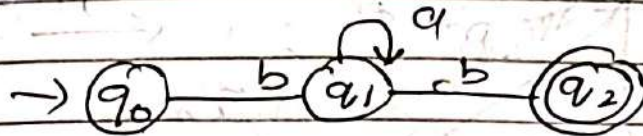
$$q_3 = [0 + (1 + 011)^* (00 + 010)]^* 1 (1 + 011)^* 01$$

Since q_3 is the final state the required expression is

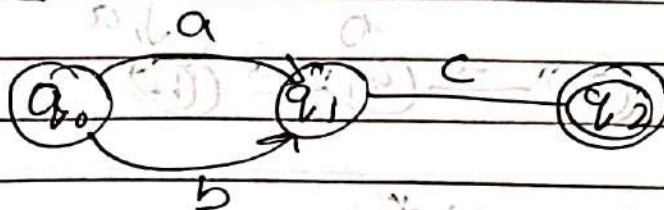
$$[0 + (1 + 011)^* (00 + 010)]^* 1 (1 + 011)^* 01$$

Convert the following RE to FA

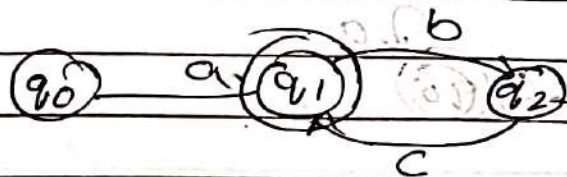
1) ba^*b



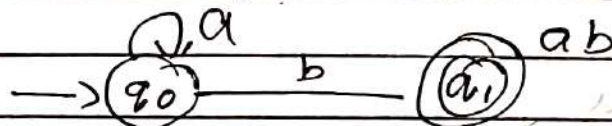
2) $(a+b)c$



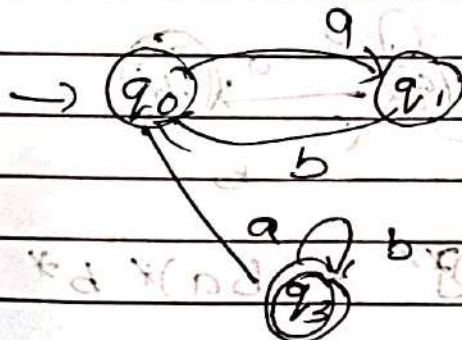
3) $a(bc)^*$



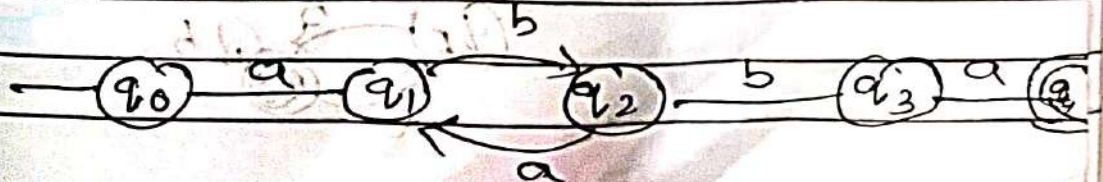
4) $a^*b(ca+b)^*$



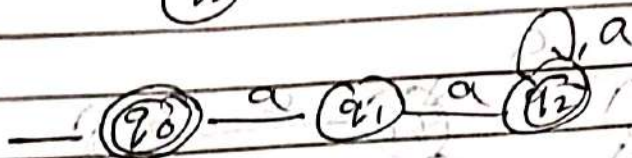
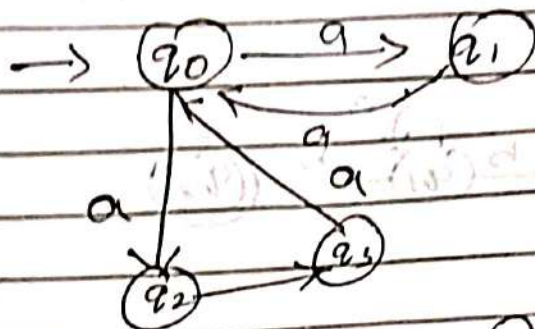
5) $(ab)^*ab^*$



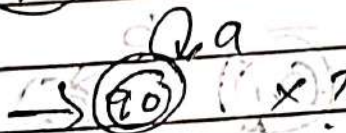
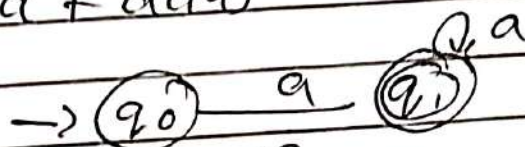
6) $(a+ba)^*ba$



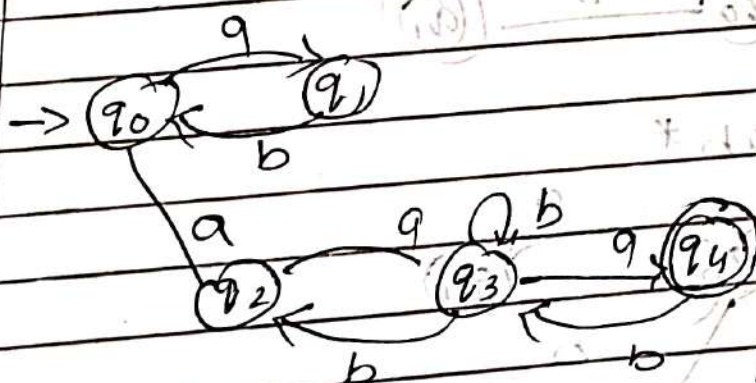
7) $(aa+aaa)^*$



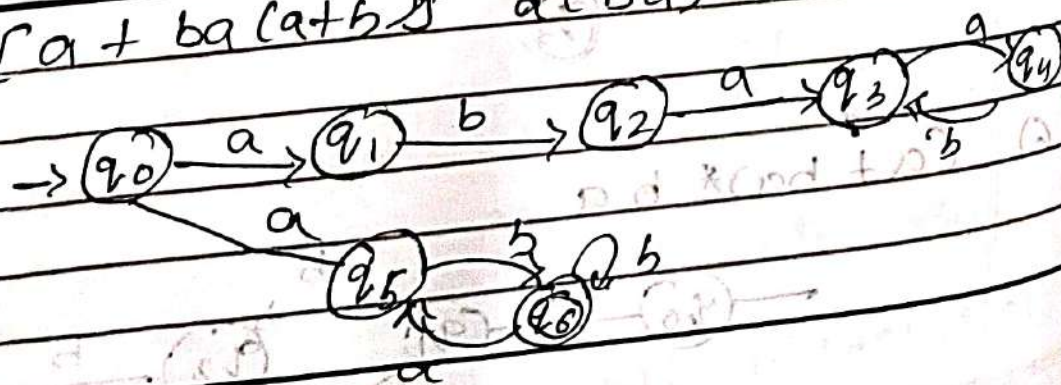
8) $(a+aaaa)^*$



9) $(ab)^* + (a+ab)^* b^* (a+b)^*$



10) $(a + ba(a+b)^* a(ba)^* b)^*$

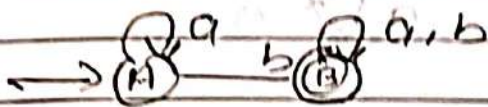


Convert the following RE to NFA

1) $a^* b (a+b)^*$

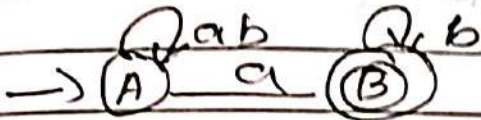
concatenation union closure

Soln



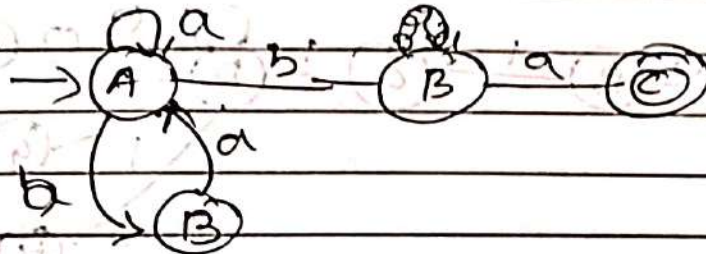
2) $(ab)^* a b^*$

Soln



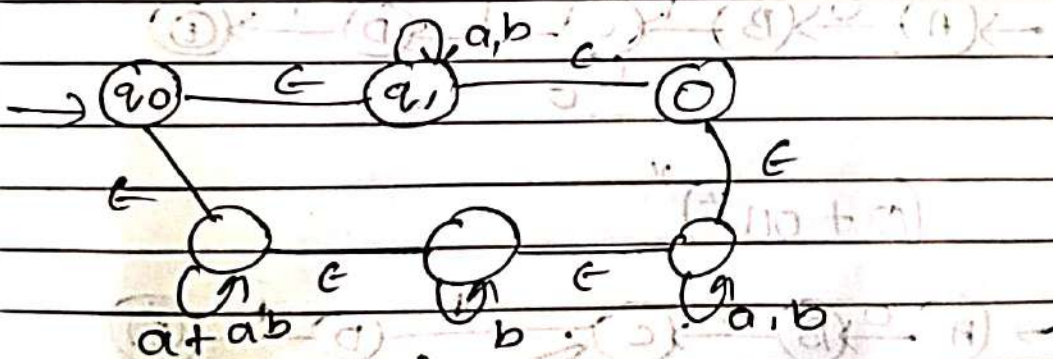
3) $(a+ba)^* ba$

Soln

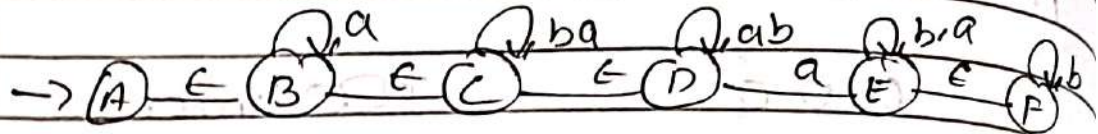
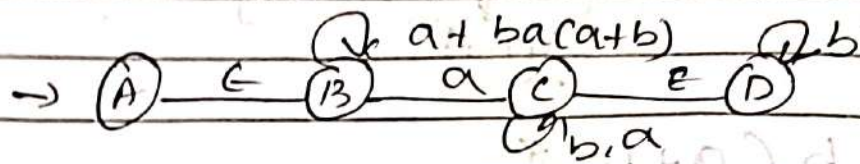


4) $(ab)^* + (a+ab)^* b^* (a+b)^*$

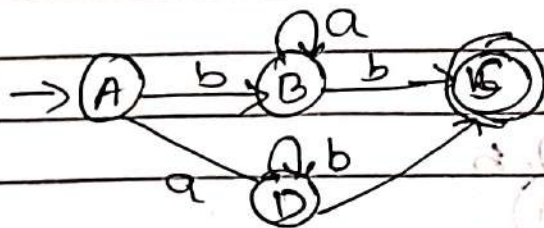
Soln



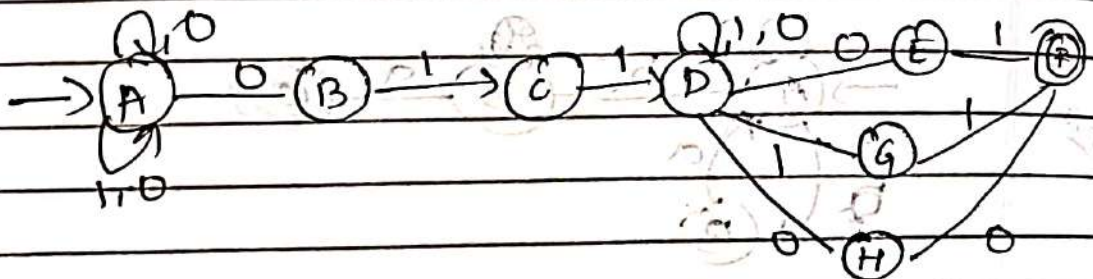
5) $(a + ba(a+b))^* a(ba)^* b^*$



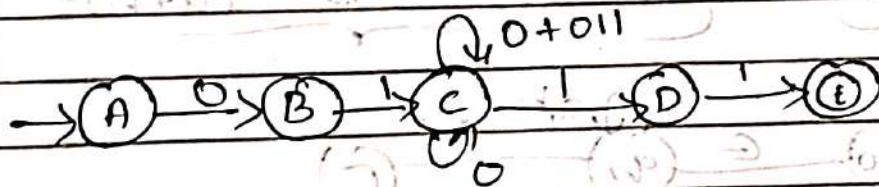
6) $b(caa)^* b + ab^* a$



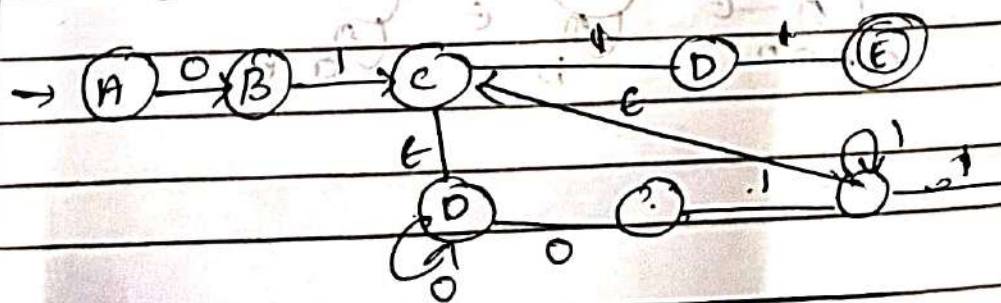
7) $(0+10)^* 011 (1+0)^* (01+11+00)$



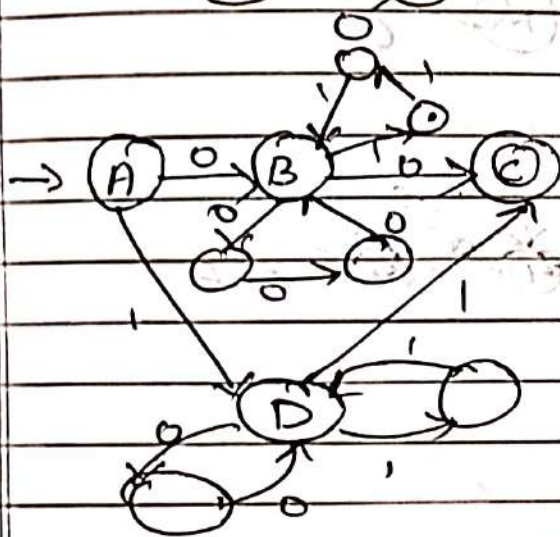
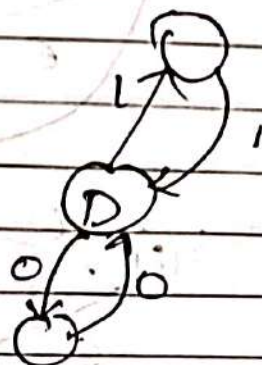
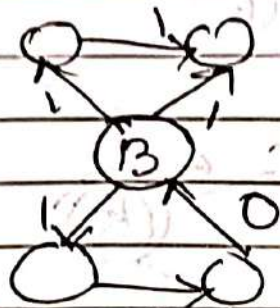
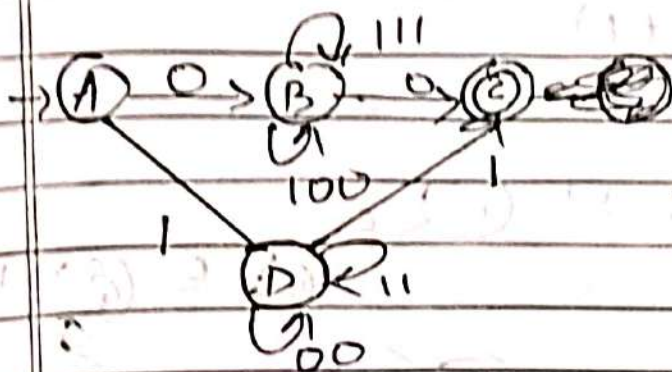
8) $01((0+011)^* + 0)^* 11$



$(0+011)^*$

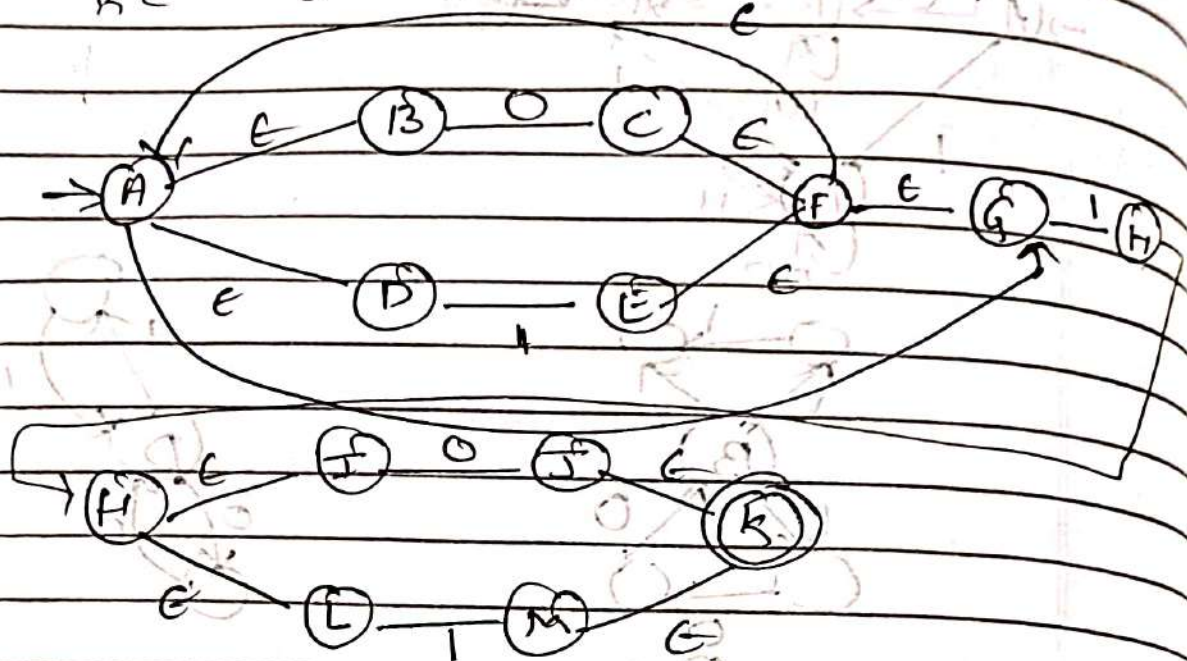


9) $0(111 + 100) \times 0 + 1(000 + 111) \times 1$

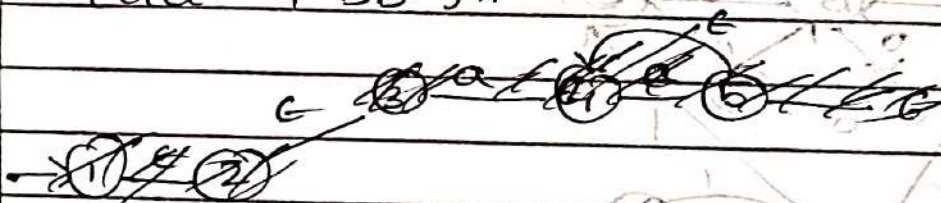


Convert the RE to E-NFA

1) $\underbrace{(0+1)^*}_{KC} \underbrace{1}_{C} \underbrace{(0+1)^*}_{on}$

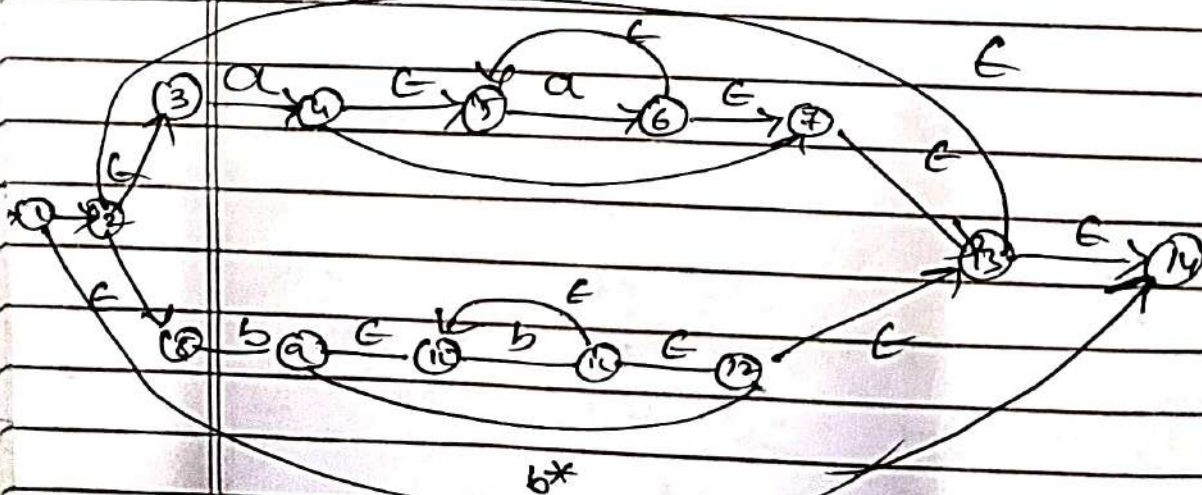


2) $(a^* + b^*)^*$

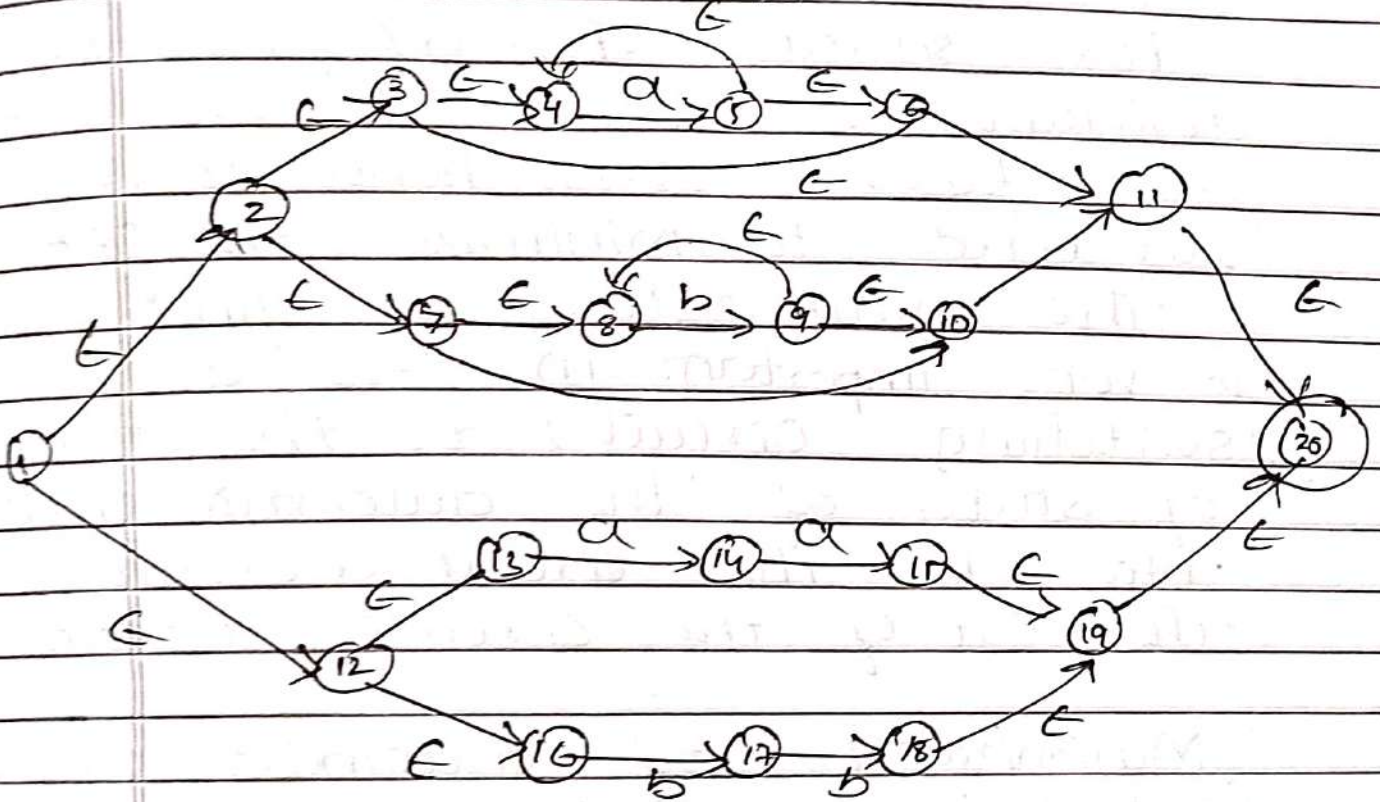


~~11 12 13~~

Create a^*



$$\underbrace{(a^* + b^*)}_1 + \underbrace{(aa + bb)}_2$$



Minimization of Finite Automata

[For storage efficiency, it is desirable to represent the DFA with fewer states], hence it is required to minimize the DFA.

[The minimization of finite automata is very important in the design of switching circuit] as the number of states of the automata implemented by the circuit decreases, the cost of the circuit also decreases.

[Minimization of automata can be achieved by finding distinguishable and indistinguishable states].

Equivalence of two states:-

[Two states p and q are equivalent (indistinguishable) if and only if $\delta(p, w)$ and $\delta(q, w)$ are final states or both $\delta(p, w)$ and $\delta(q, w)$ are non-final states for $w \in \Sigma^*$]

(i.e), if $\delta(p, w) \in F$ and $\delta(q, w) \in F$ or $\delta(p, w) \notin F$ and $\delta(q, w) \notin F$ then the states p and q are indistinguishable.

Table Filling Algorithm.

Table filling algorithm is used to find set of states that are distinguishable and indistinguishable.

The algorithm is recursively defined as follows.

Step 1: For each pair (p, q) where $p \in Q$ and $q \in Q$, if $p \in F$ and $q \notin F$ or vice versa, then the pair (p, q) is distinguishable and mark the pair by putting 'x'.

Step 2: Identify the subsequent marking. For each pair (p, q) and for each $a \in F$ Find $\delta(p, a) = r$ and $\delta(q, a) = s$.

If the pair (r, s) is already marked as distinguishable mark it as 'x'.

Repeat step 2 until no previously unmarked pairs are marked.

Step 3: If the pair (p, q) obtained using the above table filling algorithm are indistinguishable, then the two states p and q are equivalent and they can be merged into one state.

Minimization of DFA

The algorithm to minimize the DFA is as follows

Step 1: Find distinguishable and indistinguishable pairs using the table filling algorithm.

Step 2: Obtain states of minimized DFA which includes both indistinguishable pairs and distinguishable states.

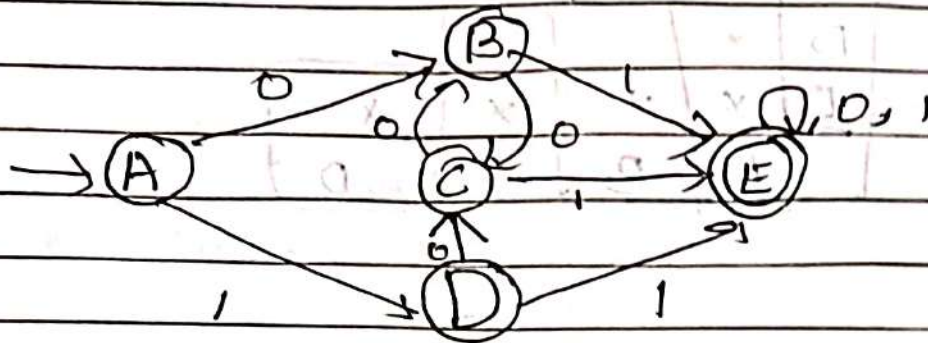
Step 3: Compute the transition table

If $[P_1, P_2, P_3, \dots, P_K]$ is a group and if $\delta([P_1, P_2, \dots, P_K], a) = [r_1, r_2, \dots, r_m]$ then place an edge from $[P_1, P_2, \dots, P_K]$ to $[r_1, r_2, \dots, r_m]$ and label the edge with the symbol a .

Follow this procedure for all states obtained in step 2 for each $a \in \Sigma$.

Problems

1) Reduce the DFA

Soln

The transition table of the above DFA

	0	1
→ A	B	D
B	C	E
C	B	E
D	C	E
* E	E	E

The DFA can be minimized using table filling algorithm

Step 1: Draw the table

3			
C			
D			
* E			
	A	B	C
			D

Initial marking

	B	x			
	C	x	.		
	D	x		.	
x	E	x	x	x	x
	A	B	C	D	

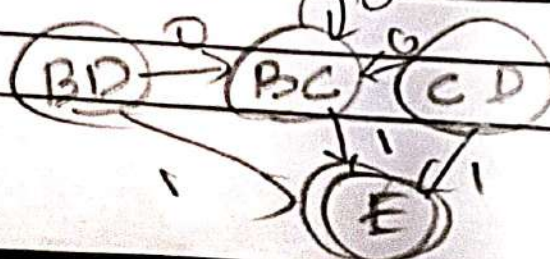
Step 2. Subsequent marking

For each pair (p, q) and for each $a \in \Sigma$ find $\delta(p, a) = r$ and $\delta(r, a) = s$

If pair (r, s) is already marked as "distinguishable" then pair (p, q) is also distinguishable and mark it as 'x'

	δ	δ	δ	
A	(A, B)	(B, C)	(C, D)	B, C not a FS
	(A, C)	(C, B)	(D, E)	D, E is final stat
	(A, D)	(B, C)	(D, E)	DE is FS/BB NFS
B	(B, C)	(C, B)	(E, E)	DE is FS/BC NFS
	(B, D)	(C, C)	(E, E)	
C	(C, D)	(B, C)	(E, E)	

Since there is no further marking stop the iteration



Indistinguishable pairs,

$(BC) (BD) (CD) = (B CD)$

Distinguishable states = A, E

Step 3: Obtain the states of minimized DFA.

A, {B, C, D}, E

Step 4: Identify the start state

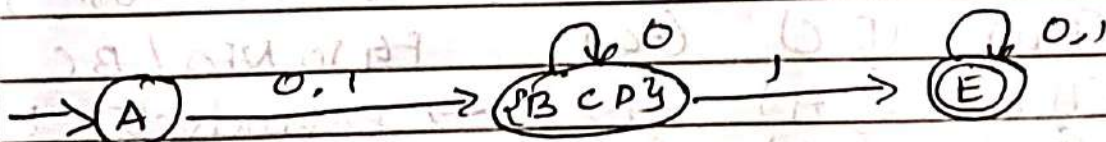
Since A is the start state of DFA, A is the start state of minimized DFA

Step 5: Identify the final state

Since E is the final state of DFA, E is the Final state of DFA

Step 6: Compute the transition table

	0	1
→ A	{B, C, D}	{B, C, D}
{B, C, D}	{B, C, D}	E
* E	E	E



2) Reduce the DFA

	S	a	b
→ A	B	E	C
B	G	C	G
x C	A	C	G
D	C	G	F
E	H	G	E
F	G	E	E
G	G	E	E
H	G	E	E

Sim

	B	x						
C	x	x						
D	x	x	x					
E	✓	x	x	x	x			
F	x	x	x	✓	x			
G	x	x	x	x	x	x	x	
H	x	✓	x	x	x	x	x	x
	A	B	C	D	E	F	G	

	S	a	b	
diff	(A.B)	(B.G)	(F.C)	FC is Fp / BG is NFA
	(A.C)	(B.A)	(F.C)	" BA is NFA
	(A.D)	(B.C)	(F.G)	FG is NFA / BC is NFA
	(A.E)	(B.H)	(F.F)	→ Biprodukt both NFA
indistinguishable	(B.H)	(G.G)	(C.C)	
	(A.F)	(B.C)	(F.G)	
	(A.G)	(B.G)	(F.G)	→ both NFA
	(B.G)	(G.G)	(C.E)	→ CE is Fp / GG NFA
	(A.H)	(B.G)	(F.C)	→ FC is Fp / BG NFA

δ	a	b
(BC)	(GA)	(CC)
(GA)	(GB)	(EF)
(GB)	GG	(EC)
(BD)	(GC)	(CG)
(BE)	(GH)	(CF)
(BF)	(GG)	(G)
(BG)	(GG)	(CE) (BH) (GG) (CC)

δ	a	b	GG (GG) (CC)
(CD)	(AC)	(CG)	
(CE)	(AH)	(CF)	
(CF)	(AC)	(CG)	
(CG)	(AG)	(CE)	A
(CH)	(AG)	(CC)	

AG $\rightarrow$ (BG) (FE) BG $\rightarrow$ (GG) (CE)

δ	a	b
(DE)	(CH)	(GF)
(DF)	(CC)	(GG)
(DG)	(CG)	(GE)
(DH)	(EG)	(GC)

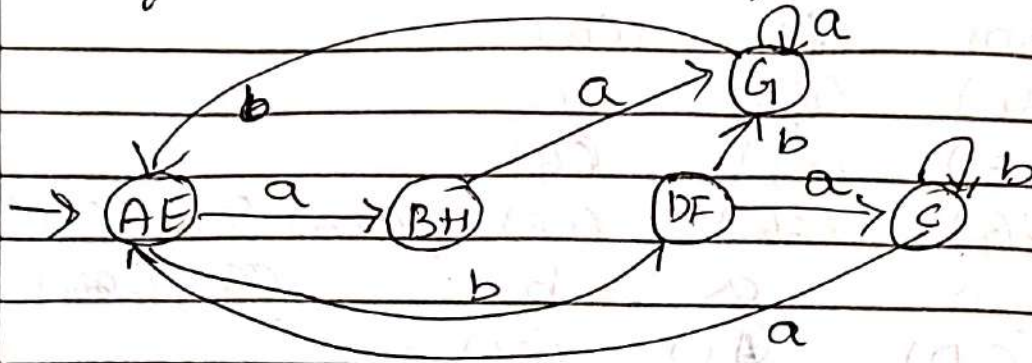
δ	a	b
(EF)	(HC)	(EG)
(EG)	(HG)	(FE)
(EH)	(HG)	(EC)

δ	a	b
(FG)	(CG)	(GE)
(FH)	(GG)	(GC)

Indistinguishable state / pairs

(AE), (BH), (DE)

left over states C, G



Regular Expressions Applications

RE are used in the following

=> Design of Compilers

=> To define languages

=> Declarative way to express set of strings

=> Validation =>

Checking the correctness of inputs

=> Tokenisation =>

Conversion of string of characters into a sequence of words for later interpretation

In pattern Matching

=> Test for a pattern within a string

=> Replace Text in a document

=> Extract a substring from a string based upon a pattern match

DATE: / /

PAGE NO: | Usha Gold

⇒ Used in languages like JavaScript and C for string handling

⇒ Helps in implementing complex match logic in databases.